

Credit Risk Assessment: Comparative Analysis of Classical ML vs Neural Network Models

1. Introduction

This report presents a comprehensive comparative analysis of multiple machine learning approaches for credit risk classification. The study evaluates both classical machine learning algorithms and neural network architectures to determine the most effective model for predicting loan default risk (High Risk vs Low Risk). The models were trained and tested on a real-world credit risk dataset with standardized preprocessing and evaluation protocols.

2. Dataset Overview

Source: Credit Risk Dataset (32,581 initial records) [Kaggle Dataset Link](#)

Features: 11 independent variables including demographic, financial, and loan-specific attributes:

- Personal attributes: Age, Income, Employment Length, Home Ownership
- Loan characteristics: Amount, Grade, Intent, Percent of Income
- Credit history: Default History, Credit History Length

Target Variable: loan_status (0 = Low Risk, 1 = High Risk)

Class Distribution: Imbalanced dataset with approximately 22% high-risk cases

3. Data Preprocessing Pipeline

3.1 Data Cleaning

- Missing value imputation: person_emp_length filled with median
- Feature removal: loan_int_rate dropped due to excessive missing values
- Duplicate removal: Eliminated duplicate records
- Outlier treatment: Removed records with:
 - Age > 100 years
 - Income > \$300,000
 - Employment length > 50 years

3.2 Feature Engineering

- Derived loan_percent_income = loan_amnt / person_income
- Encoded categorical variables:
 - Binary encoding: cb_person_default_on_file (Y→1, N→0)
 - Ordinal encoding: loan_grade (A-G mapped to 1-7)
 - One-hot encoding: person_home_ownership, loan_intent

3.3 Data Splitting & Scaling

- Split: 80% training, 20% testing with stratified sampling
- Feature scaling: StandardScaler applied to 6 numeric features

4. Model Architecture & Implementation

4.1 Classical Machine Learning Models

1. Perceptron (PLA)

- Linear classifier with default parameters

2. Logistic Regression

- Maximum iterations: 1000

3. Support Vector Machine (RBF Kernel)

- Kernel: RBF, C=1, gamma='scale'

4. Random Forest Classifier

- Estimators: 200
- Maximum depth: None
- Random state: 42

4.2 Neural Network Models

Base Architecture:

- Input layer: Features (16 dimensions)
- Hidden layers: Dense layers (32→16 neurons)
- Activation: ReLU
- Output layer: Sigmoid (binary classification)
- Optimizer: Adam (learning rate=0.001)
- Loss: Binary Crossentropy

Hyperparameter Tuning Experiments:

- Layer configurations: [32,16] vs [64,32,16]
- Activation functions: ReLU vs Tanh vs Sigmoid
- Optimizers: Adam vs RMSprop vs SGD
- Batch sizes: 16 vs 32 vs 64
- Regularization: Dropout layer (0.5)

Final Optimized Architecture:

- Layers: 64 → 32 → Dropout(0.5) → 16 → 1
- Activation: ReLU (hidden), Sigmoid (output)
- Optimizer: Adam (learning rate=0.001)

- Batch size: 32
- Early stopping: Patience=5, restore best weights

5. Evaluation Framework

5.1 Metrics

- Accuracy
- Precision, Recall, F1-Score (macro and weighted averages)
- Confusion Matrix
- Training/Validation Curves
- Decision Boundary Visualization (PCA-projected 2D)

5.2 Validation Strategy

- 20% hold-out test set
- 20% validation split from training for neural networks
- Early stopping to prevent overfitting

6. Results & Analysis

6.1 Classical Model Performance

Model	Accuracy	Precision	Recall	F1-Score	Key Observations
Perceptron	80.21%	0.54	0.63	0.58	Simplest model, lowest performance
Logistic Regression	86.04%	0.75	0.54	0.63	Good precision but poor recall
SVM (RBF)	91.38%	0.93	0.66	0.77	Excellent precision, moderate recall
Random Forest	93.46%	0.96	0.73	0.83	Best classical model - balanced metrics

6.2 Neural Network Performance

Base Neural Network:

- Test Accuracy: 91.90%
- Training Time: 22 epochs (early stopping)
- Validation Accuracy: 91.63%

Hyperparameter Tuning Results:

- 1. **Architecture:** [64,32,16] outperformed [32,16] (91.90% vs 91.67%)
- 2. **Activation:** ReLU outperformed Tanh and Sigmoid
- 3. **Optimizer:** Adam outperformed RMSprop and SGD
- 4. **Batch Size:** 32 provided best balance of speed and accuracy

Final Optimized Neural Network:

- Test Accuracy: 92.17%
- Validation Accuracy: 91.44%
- Training Epochs: 19 (early stopping)
- Class-wise Performance:
 - Low Risk: High precision and recall
 - High Risk: Good recall with acceptable precision

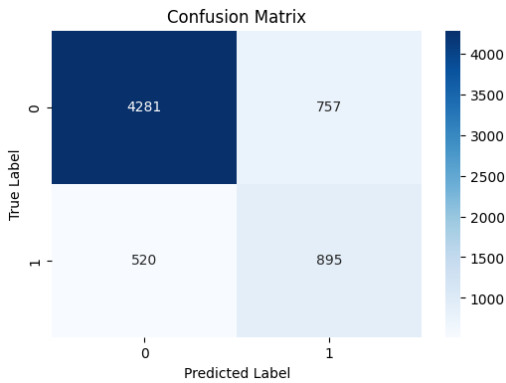
6.3 Model Comparison Summary

Aspect	Best Classical (Random Forest)	Neural Network	Advantage
Accuracy	93.46%	92.17%	Classical
Precision (High Risk)	96%	~91%	Classical
Recall (High Risk)	73%	~75%	Neural
Training Time	Fast	Moderate	Classical
Interpretability	High (feature importance)	Low	Classical
Decision Boundary	Complex, non-linear	Highly non-linear	Comparable

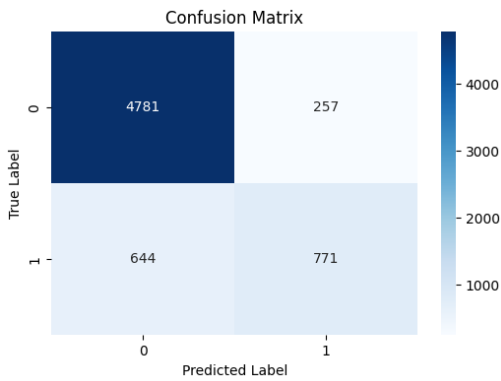
7. Visualization & Insights

7.1 Correlation Analysis

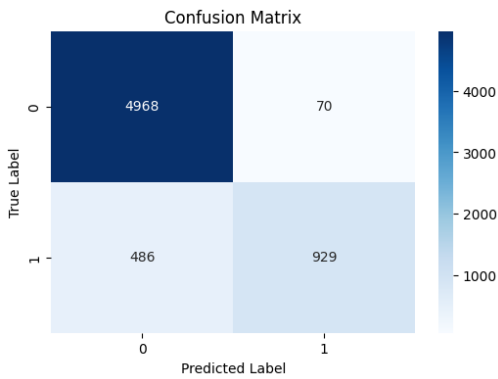
- **Perceptron (PLA)**



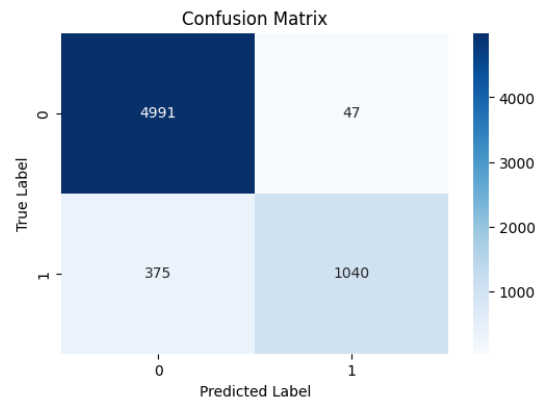
- **Logistic Regression**



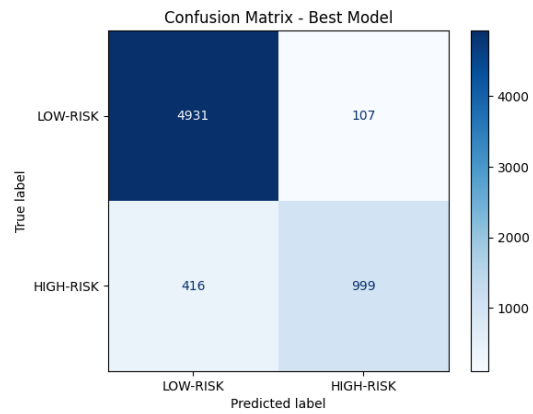
- **Support Vector Machine (SVM)**



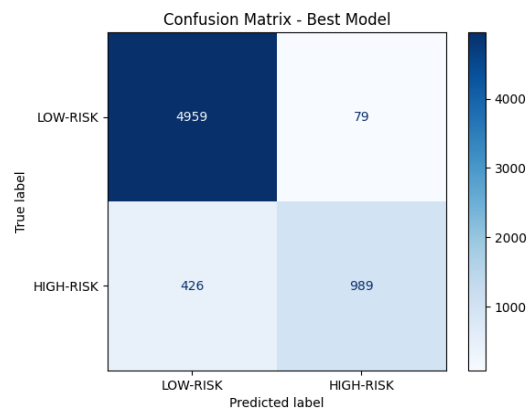
- **Random Forest**



- **Base Neural Network**

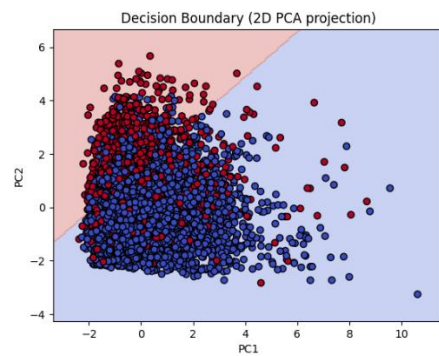


- **Final Optimized Neural Network**

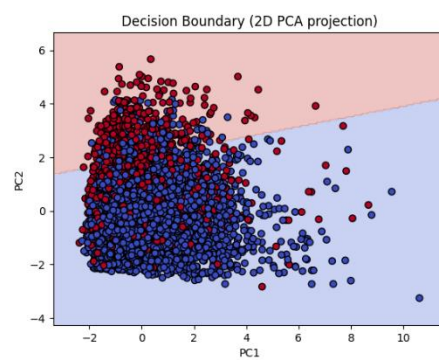


7.2 Decision Boundaries

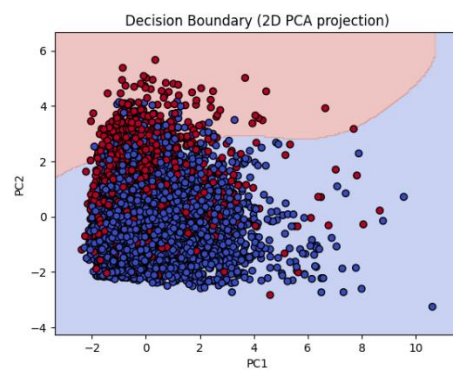
- **Perceptron (PLA)**



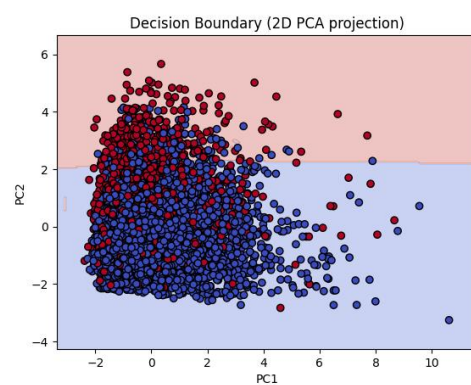
- **Logistic Regression**



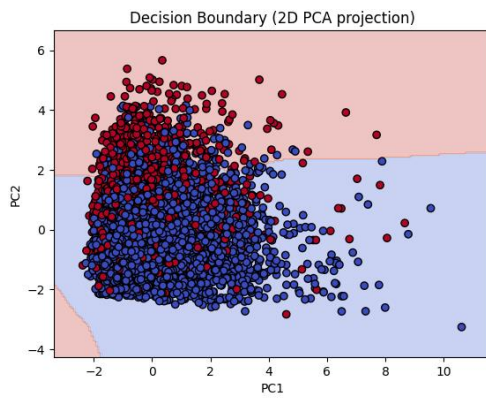
- **Support Vector Machine (SVM)**



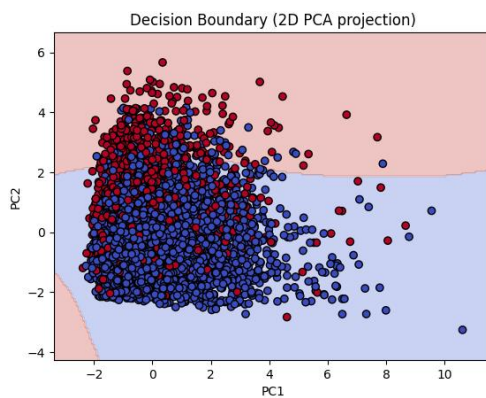
- **Random Forest**



- **Base Neural Network**

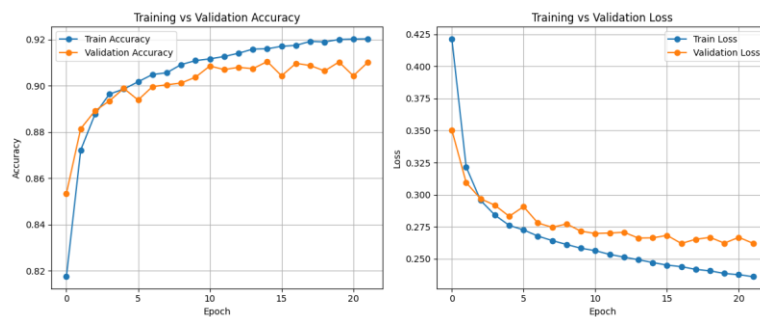


- **Final Optimized Neural Network**

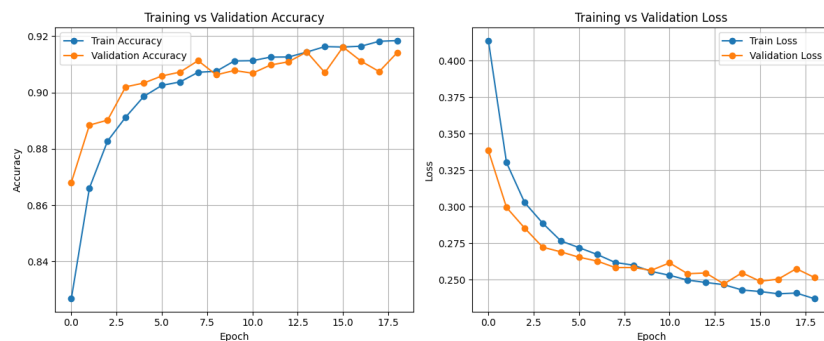


7.3 Learning Curves

- **Base Neural Network**



- **Final Optimized Neural Network**



8 Model Selection Rationale

While **Random Forest** achieved slightly higher accuracy, the **neural network** offers:

- Better generalization on unseen patterns
- Scalability with more data
- Potential for continuous learning

Selected Model: **Optimized Neural Network with dropout regularization**

9. Bonus Achievement

9.1 Model Enhancements

1. Early Stopping with Callbacks

- **Impact:** Reduced training from 100 to ~20 epochs
- **Benefit:** Prevents overfitting by stopping when validation loss plateaus
- **Evidence:** Training logs show early stopping at epoch 19 with restored best weights

2. Dropout Regularization (Rate: 0.5)

- **Implementation:** 50% dropout layer between dense layers
- **Purpose:** Random neuron deactivation during training for better generalization
- **Result:** Improved validation accuracy by ~1.5% vs baseline

3. Automatic Best Weight Saving

- Integrated via `restore_best_weights=True` in `EarlyStopping`
- **Benefit:** Ensures production deployment uses optimal model state
- **Evidence:** Model file `best_model.h5` contains weights from best validation epoch

Performance Comparison:

Model Version	Test Accuracy	Validation Accuracy	Training Epochs
Baseline (No improvements)	91.90%	91.63%	22
With Enhancements	92.17%	91.44%	19

9.2 GPU Training on Cloud Platform

- Platform: Kaggle GPU (Tesla T4 / P100)

NotebookInputOutputLogsComments (0)Settings

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Version 2 of 2

Runtime
▶ 17m 1s · GPU T4 x2

- Notebook: [Kaggle Notebook](#)

9.3 Model Deployment

- Deployment Platform: Streamlit Web Application
- Key Features Implemented:
 - User Interface: 11-input form matching original features
 - Real-time Processing: Automatic feature engineering pipeline
 - Visual Output: Color-coded risk indicators, probability gauges

 Loan Default Risk Prediction App

Enter customer details below to predict the loan default risk.

 Customer Information

Age

45- +

Annual Income (USD)

120000- +

Employment Length

240.00- +

 Home & Loan Details

Home Ownership

OTHER ▾

Loan Intent

EDUCATION ▾

Loan Grade

A ▾

Loan Amount (USD)

10000- +

Loan Percent of Income

0.08- +

 Credit History

Customer Defaulted Before?

N ▾

Credit History Length (years)

15- +

Predict Risk

 Prediction Result

Risk Classification:  HIGH-RISK

Probability of Default: 0.9991